

8A : Basic I/O Concepts

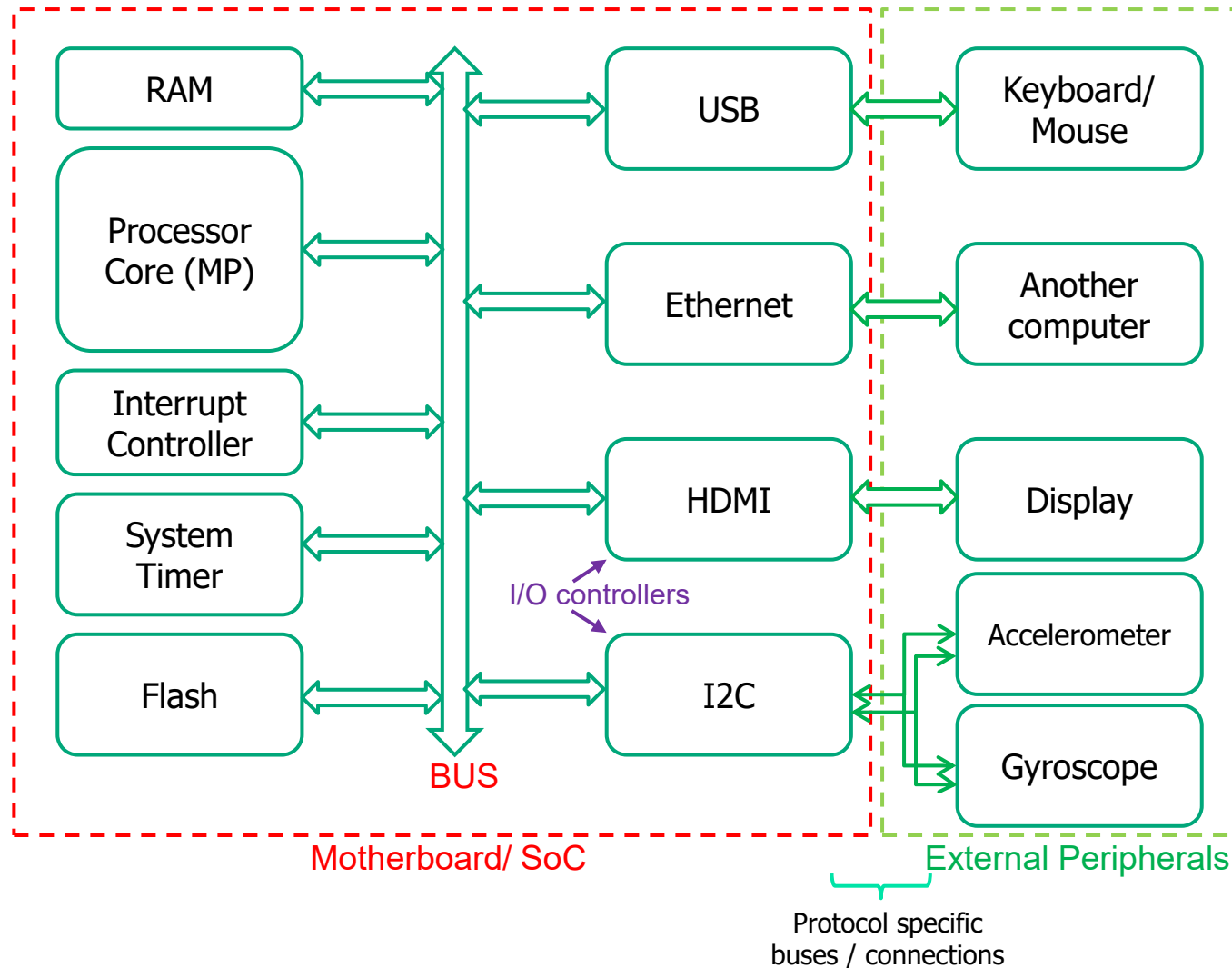
Note :

- Slides not covered in detail in the class are left as a self-learning exercise

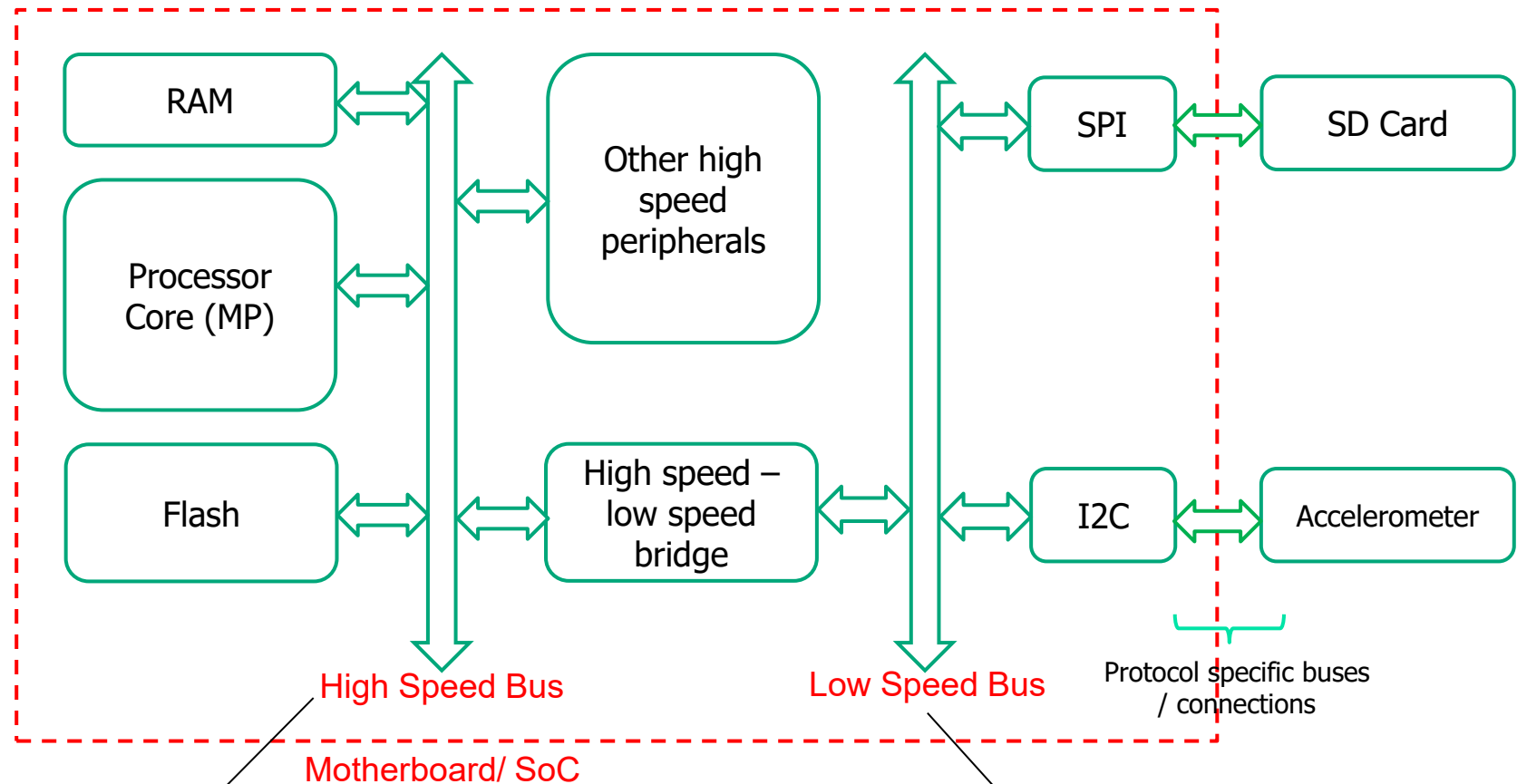
Acknowledgement :

- Relevant Wikipedia articles
- Text by Harris and Harris and companion materials

Generic Computer System



Multi-tiered Bus Architecture

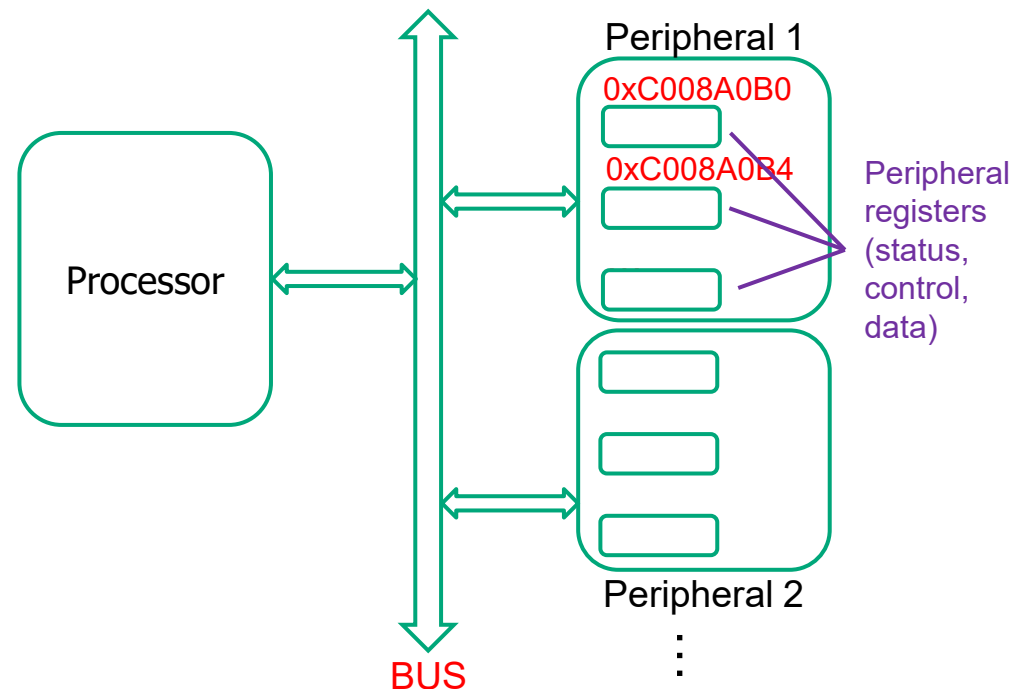


Usually proprietary such as
QuickPath Interconnect (Intel),
Hypertransport (AMD), AHB* (ARM)
*AHB is an open standard

Usually industry standard
such as PCIe, APB/AXI

Peripheral Registers

- Software interacts with the hardware by reading / writing peripheral registers
- Peripheral registers are accessed like memory – using LDR/STR or pointers for peripherals within the address space of the processor
- Status registers - *read* by the processor to know the status of the hardware
- Control registers - *written* by the processor to give commands / control signals
- Data registers - *read* (for input) and/or *written* (for output) to perform actual transfer of data





C language, libraries / API / drivers

- Libraries (collection of driver functions for a peripheral / API) encapsulate the process of reading/writing peripheral registers
- Internally use pointers to peripheral registers (i.e., holding peripheral register addresses) to accomplish reading/writing
- Allow ease of programming (higher layer of abstraction – no need to reinvent the wheel)
- Portability – we just need to recompile / re-link with the new library, user program will need minimal/no changes



Programmed I/O (PIO)

How exactly does the data transfer take place?

- Programmed I/O (PIO)

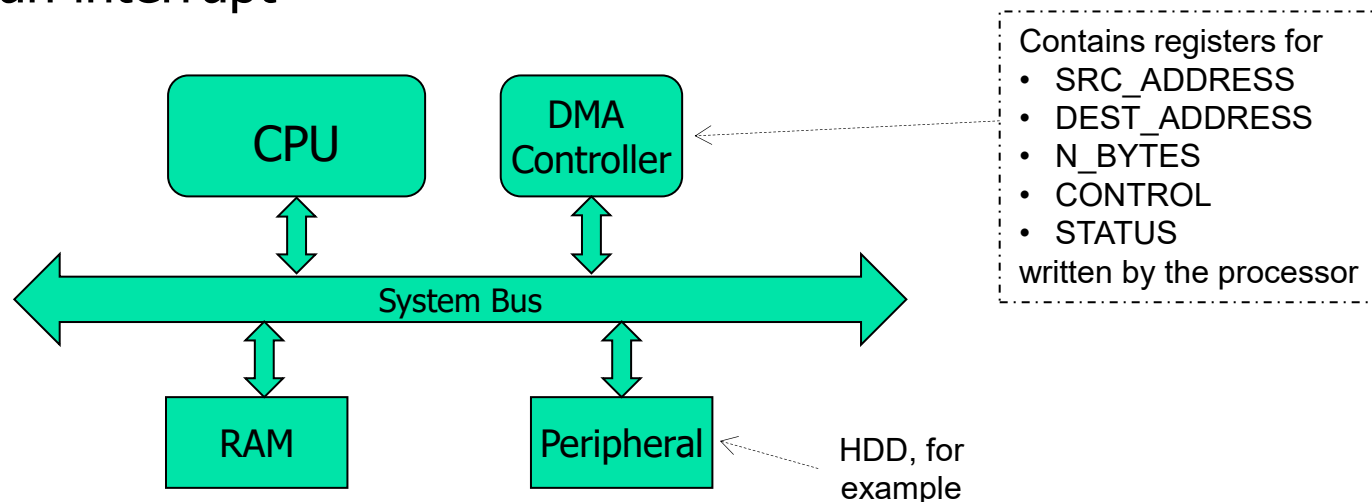
- The processor reads the peripheral register and places the data into a location in the memory (or vice versa)
- Done using a combination of load-store instructions
- Access initiated through polling or triggered by an interrupt
- Problem : Processor time is wasted for bulk transfers
- Solution : Direct Memory Access

- DMA

- A separate controller is present on the bus, capable of generating addresses, bus signals (R/W etc.) and transferring data
- CPU's job is limited to initiating the data transfer

Direct Memory Access (DMA)

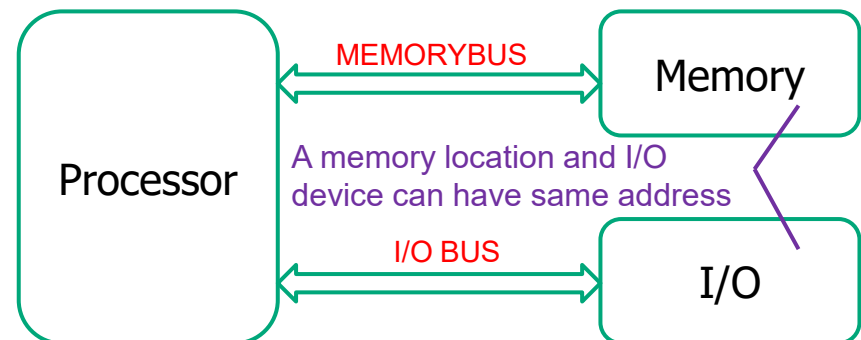
- CPU (OS) provides the DMA controller the source address, destination address and the number of bytes/words to be transferred
- CPU then assigns DMA as the bus master
- DMA controller generates the addresses sequentially and performs the transfer
- In the meantime, the CPU can perform computations which do not need memory access (using the data in cache and registers)
- The DMA controller informs the CPU once the transfer is over by raising an interrupt



Port-mapped I/O

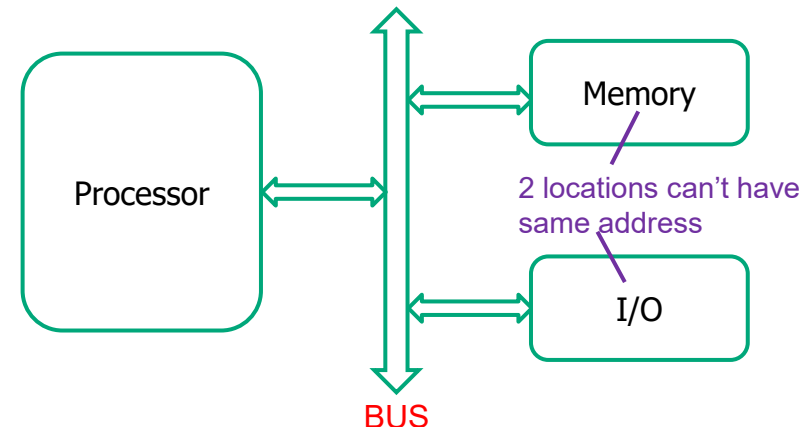
How can the processor address an I/O device/register?

- Port-mapped I/O (PMIO, also called isolated I/O)
 - Dedicated instructions for I/O
 - Ex : Intel x86 instruction set has dedicated IN and OUT instructions, separate from memory access instructions
 - The processor has an output pin specifying whether the address is meant for memory or peripheral
- ④ Separate address space for memory and peripherals - memory can use the full range of addresses
- ④ Simpler software
- ④ Not compliant with RISC philosophy
- ④ Extra pins



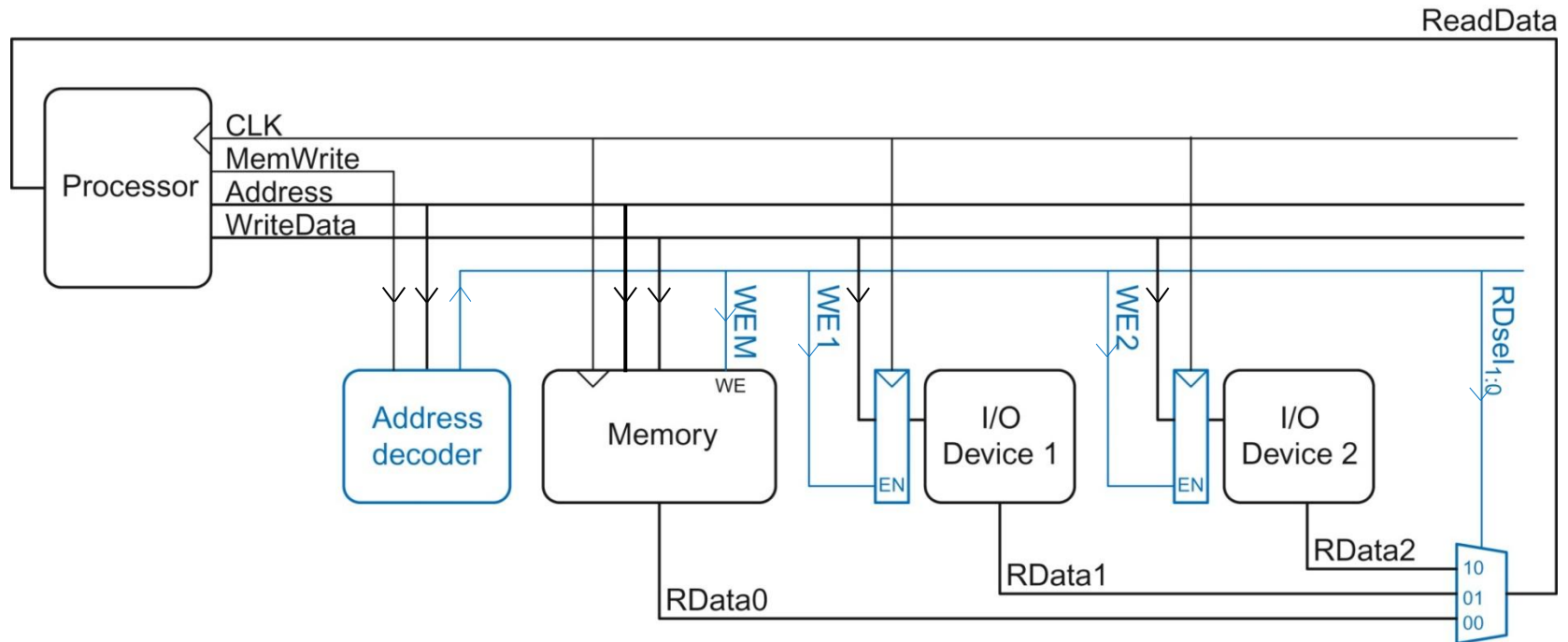
Memory-mapped I/O

- Memory-mapped I/O (MMIO)
 - Same LDR and STR instructions used for I/O, just like memory
 - Address space is shared between memory and I/O
 - An address decoder determines whether memory or I/O, and which I/O device
- ① Processor design is simpler
- ① Maximum possible memory is reduced, as I/O devices eat into the address space – not a problem with 64-bit systems, but an issue for 32-bit systems
- ① Software is more complicated
 - ② Peripheral registers do not generally behave like memory – they are not simply storage locations!



MMIO : Hardware

- This is the mechanism we used in our labs
 - Go through the Wrapper.v/vhd file!



MMIO – S/W Complications

- Software needs to flush the (write-back) cache immediately after writing
 - To force the data to be written to the peripheral register
- Should do cache invalidation before reading
 - To ensure that we read the updated value from the peripheral register rather than the cached value
- Some cache controllers support designating MMIO memory range as non-cacheable
- `volatile` type qualifier necessary

```
*UART_ADDR = var1; //writes to the transmitter
var2 = *UART_ADDR; // reads from the receiver
```

 - Without `volatile`, the optimizing compiler will replace the above with `var2 = var1;` which is WRONG!
 - Transmit register (write only) and receive register (read only) happens to have the same address but are physically not the same!

Note: `volatile` is also necessary for variables modified by an interrupt handler (see later) 11

MMIO – S/W Complications

- An out of order (OoO) processor could rearrange

```
*UART_ADDR = var1; // writes to the transmitter (data register)  
var2 = *UART_OUT_READY_ADDR; // reads from the  
transmitter ready (status) register
```

- Unlike memory, `UART_ADDR`, `UART_OUT_READY_ADDR` are not independent of each other though they have different addresses!
- `UART_OUT_READY` becomes 0 immediately after `UART` is written, and becomes 1 after the byte has been transmitted out and the transmitter is ready for the next byte
- `volatile` prevents compiler reordering; doesn't stop OoO
- Needs a memory barrier (`fence` instruction in RISC-V) about which rearrangement is not possible
 - Typically inserted in C using inline assembly macros, like `mb()` in Linux kernel
 - `fence` has some variants depending on which all operations can't be rearranged (reads aka loads, writes aka stores, both)

IO – Atomicity

- Suppose the LEDs currently display (0b) 00010100. We wish to change it to 00110100. This can be done via:

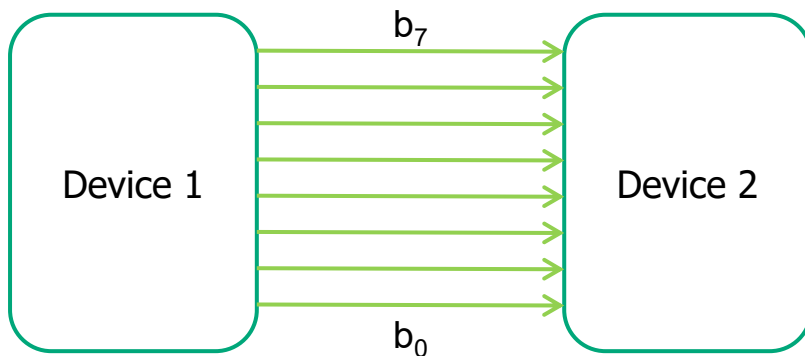
```
tmp = *LED_ADDR;  
tmp = tmp | 00100000;  
*LED_ADDR = tmp;
```

(assuming LED output register can be read back – the Wrapper in CG3207 labs do not allow this, but can be modified to allow it very easily)

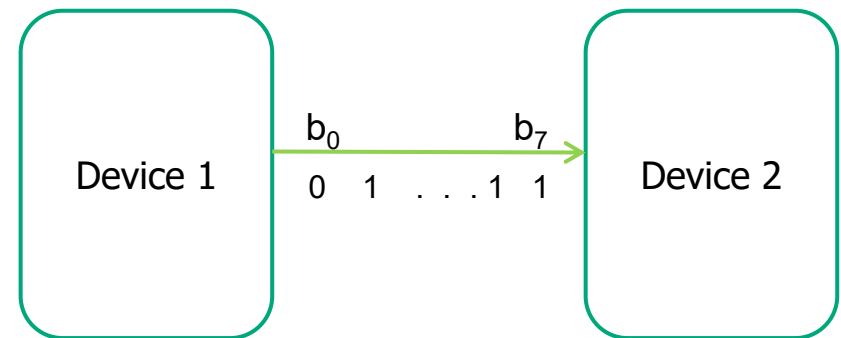
- Say, an interrupt occurs between the second and third operations above, and the interrupt handler modifies LEDs
- The third line above overwrites any modification done by the handler, effectively destroying the information from the interrupt
- This can be prevented by doing the above using an *atomic* ('A' extension in RISC-V ISA) instruction `amoor.w`, which does read-modify-write in a non-preemptible manner (won't execute partially, can't be interrupted in the middle)
- Atomic instructions are also used to implement mutexes, etc. when done on memory locations. Also needed for PMIO, not just MMIO

Protocol Classification - Parallel vs Serial

- Parallel protocols : use a number of parallel wires to transmit bits in parallel
 - Processor-to-memory busses, AXI, AXI Stream, old (pre-2005) protocols such as PCI, IDE, printer port
- Serial protocols : data is sent through a single wire, one bit at a time
 - Serial ATA (SATA), PCIe, I²C, SPI, UART, CAN



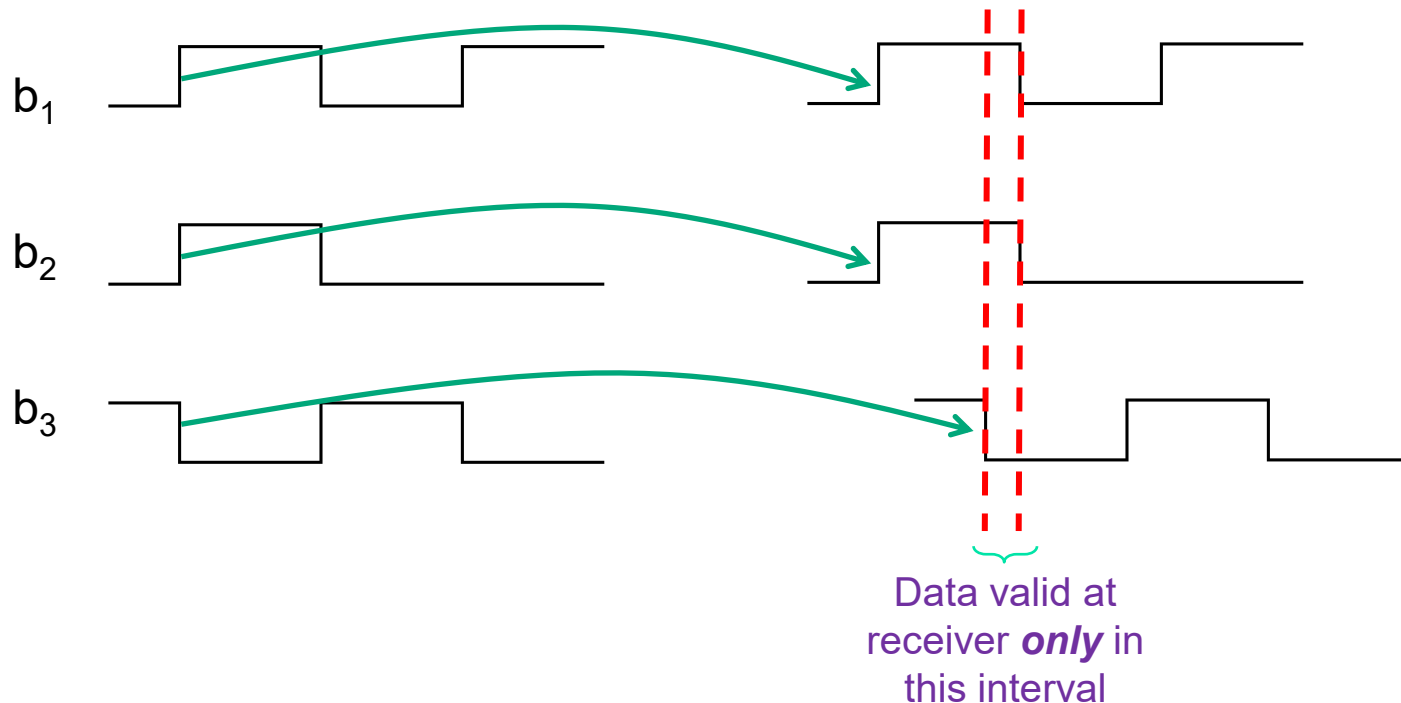
Need not always be 8-bit



MSB First (Need not always be the case)

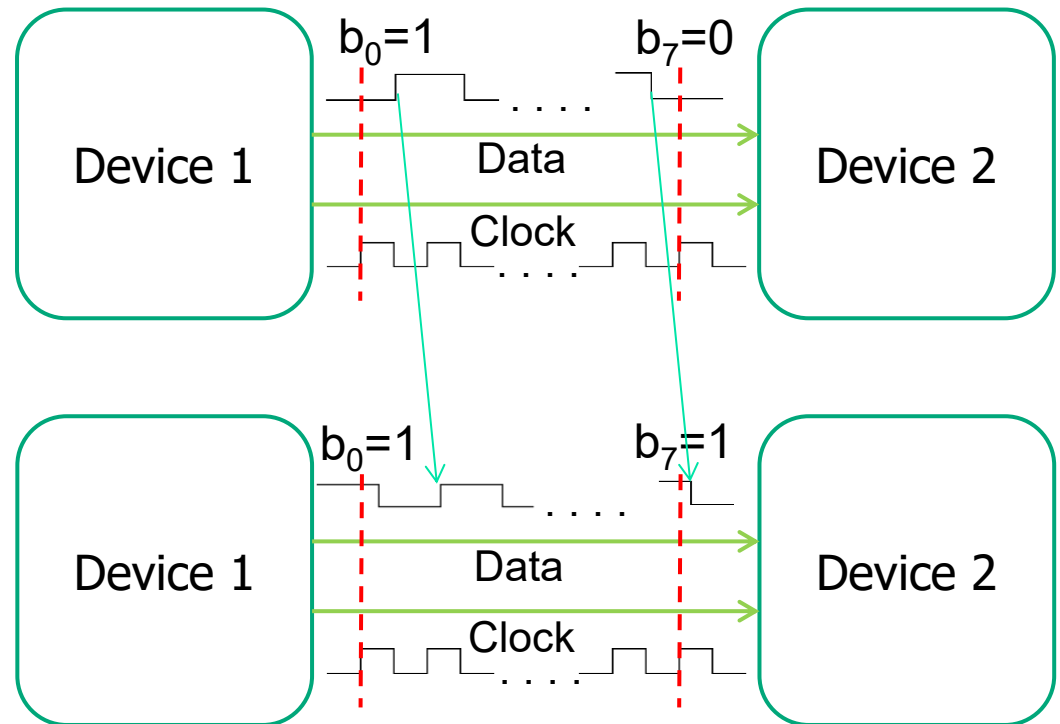
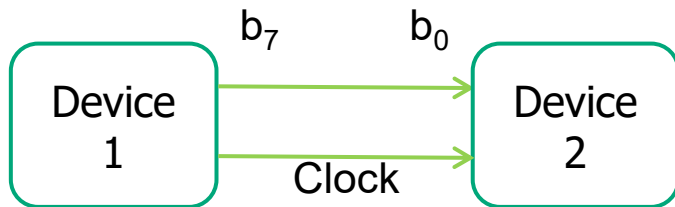
Parallel Buses : Bus Skew

- Parallel : used in high-speed, low distance
- Suffers from bus-skew : different signals travel at diff. speeds
- Suffers from cross-talk : signals from diff. wires interfere with each other



Synchronous vs Asynchronous

- Synchronous : one of the wires carry clock - allows transmitter and receiver to have a common time reference
- Faster, but suffers from the same issue as the parallel protocols (cross-talk, skew etc.)
 - SPI, I²C, AXI, AXI Stream



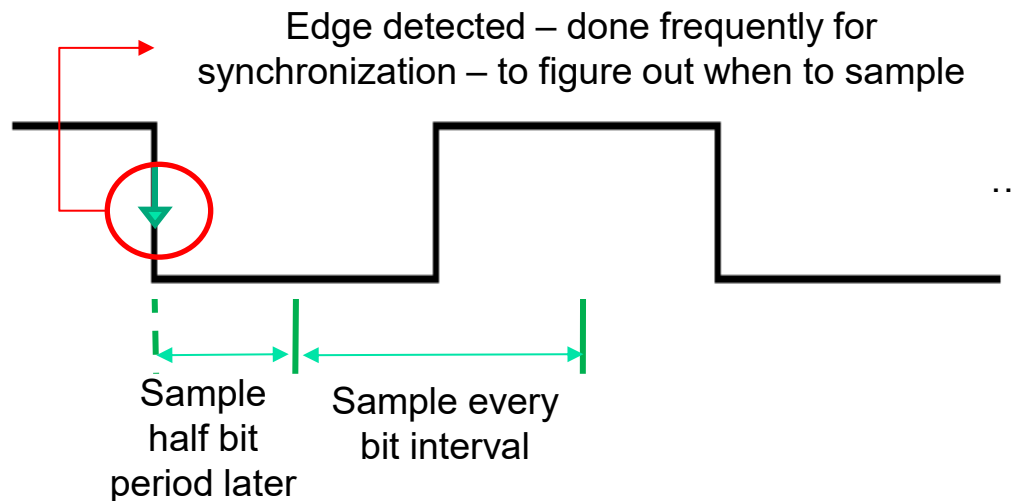
Note: Clock need not always be generated by the transmitting device

Synchronous vs Asynchronous

- Asynchronous : only data is transmitted, not clock
- Receiver needs to recover timing info from data – slower, complicated hardware
 - UART, USB*, CAN



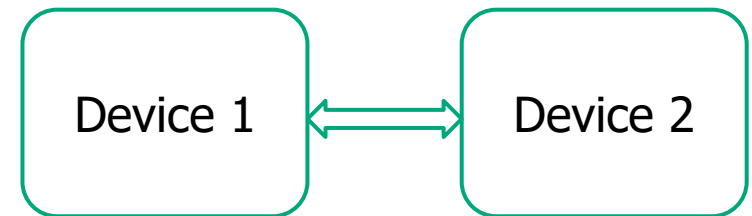
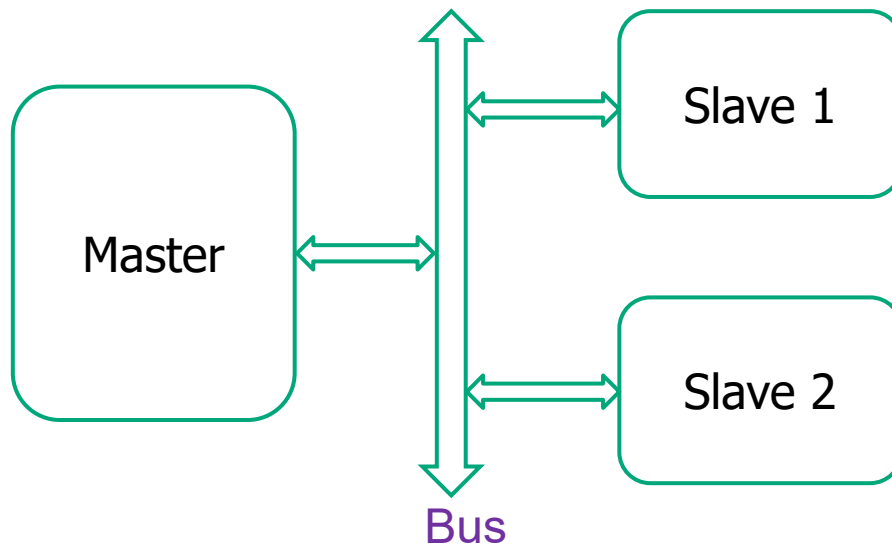
A handshaking mechanism with additional control lines may also be used instead of edge detection based scheme. Here, the transmitting party will assert a 'ready' signal and keeps the data unchanged until it receives an 'acknowledgement' from the receiving party.



*perspective dependent. While the transmitter and the receiver do not share a clock explicitly, the clocks are synchronized through sync fields and remain in sync through appropriate coding of data

Bus-based vs Point-to-point

- Bus : a number of devices are connected to the same set of wires (called a bus) – each device has a unique address
 - I²C, SPI, USB, AXI, CAN
- Point-to-point : two devices have a dedicated link between them – no addressing
 - UART, AXI Stream



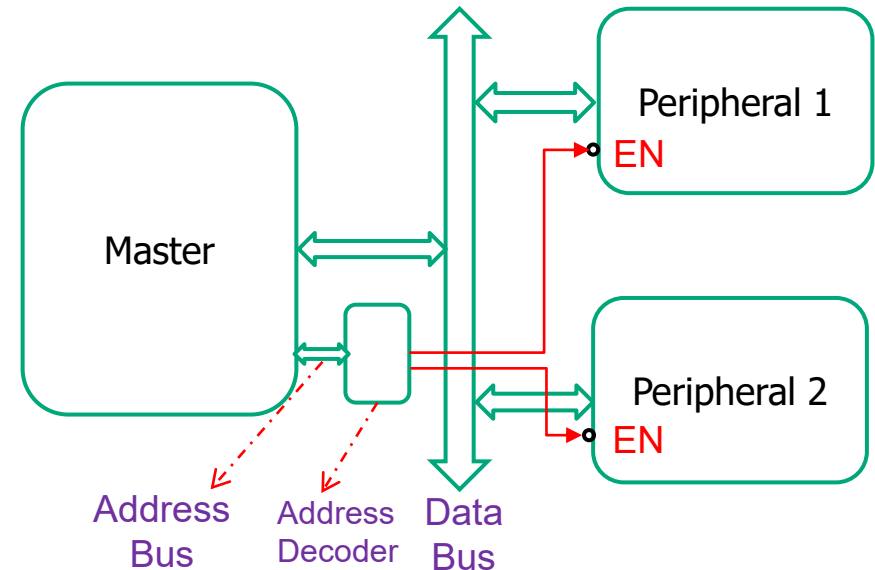
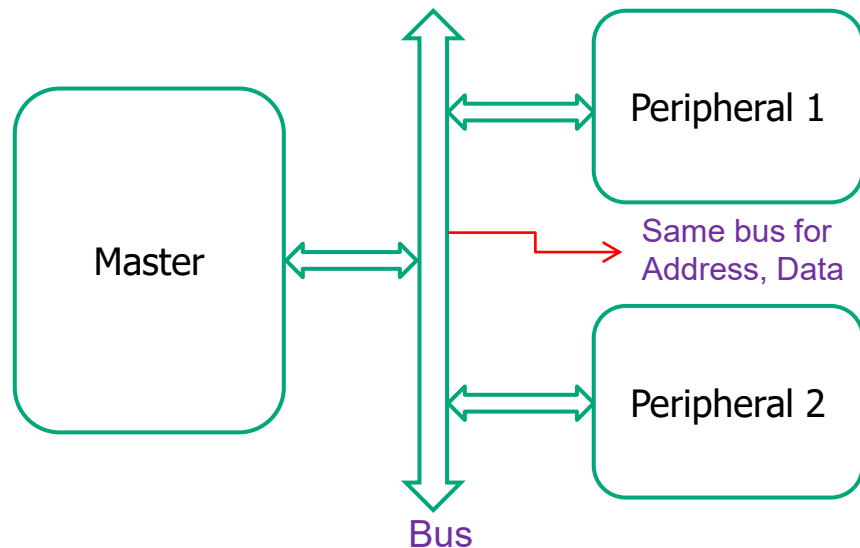
Master-Slave vs Peer-to-peer

- Master-slave : only a master device can initiate communication, decides which slave should respond, and (in some synchronous protocols) generates the clock
 - I²C, SPI, USB, AXI, AXI Stream
- Peer-to-peer : any device can initiate communication
 - Ethernet (configuration dependent)



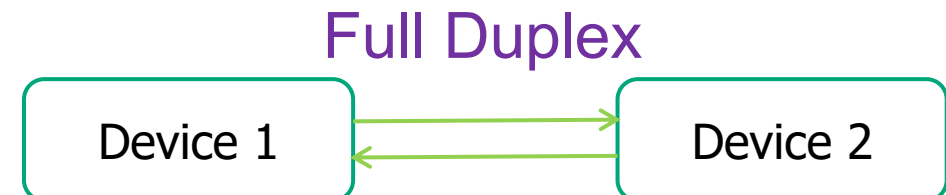
Addressing : In-band vs Out-of-band

- In-band : address of the device being accessed (to select the device), as well as the data (read from or written to the device) are sent through the same bus
 - I²C, USB
- Out-of-band : separate address bus, a decoder enables (activates) the device
 - SPI



Simplex vs Half Duplex vs Full Duplex

- Simplex : single, unidirectional link, one-way interaction
 - Connection between processor and LEDs / switches, AXI Stream
- Half-duplex : one bi-directional link, devices take turns
 - I²C, USB 2.0
- Full-duplex : at least two links, can transmit and receive at the same time
 - SPI, UART, USB 3.x

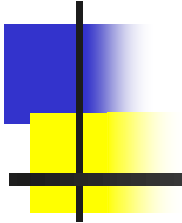


To do: Read about the various protocols that are mentioned as examples under the various categories in the last few slides!



Summary

- DMA relieves CPU of the burden of data transfer
- Software for I/O has to be designed carefully – issues related to
 - Cache
 - Compiler optimization
 - Out of order processing
 - Need for atomicity
- A number of different classification criteria exists to classify protocols



8B : Exception Handling

Note :

- Slides not covered in detail in the class are left as a self-learning exercise

Acknowledgement :

- Relevant Wikipedia articles, ChatGPT responses
- Text by Harris and Harris and companion materials

Dealing with Exceptions

- I/O devices tend to be very slow relative to the processor
 - Polling I/O devices generally wasteful, hence use interrupt
- Exception is a mechanism to wrestle back control from a non-privileged program – for security, task management, etc.
- A way to deal with 'situations' that may arise during execution
- A way for application programs to invoke the OS
- Can be used to simulate instructions not supported by hardware
- Exceptions (aka interrupts in a generic sense) are just another form of control hazard. Exceptions arise from
 - DP arithmetic errors
 - Trying to execute an undefined instruction
 - An I/O device request
 - An OS service request (e.g., a page fault, TLB exception)
 - A hardware malfunction

Dealing with Exceptions

- When an exception happens, the pipeline has to stop executing the offending instruction in midstream, let all prior instructions complete, flush all following instructions, (sometimes) set a register to show the cause of the exception, save the address of the offending instruction, and then jump to a prearranged address (the address of the exception handler)
- The handler looks at the cause of the exception and “deals” with it. The address of the handler is usually found in a table addressed by a control register, e.g., `mtvec` in RV
- In some processors, there are status registers that record the ‘cause’ of exception, e.g., `mcause` in RV
- The address of the instruction that caused the exception / that did not complete because of the exception is also recorded, e.g., `mepc` in RV (`mepc` = machine exception program counter)
- `crr` / `crrw` instructions used to read/write the control and status registers (CSRs) such as those mentioned above

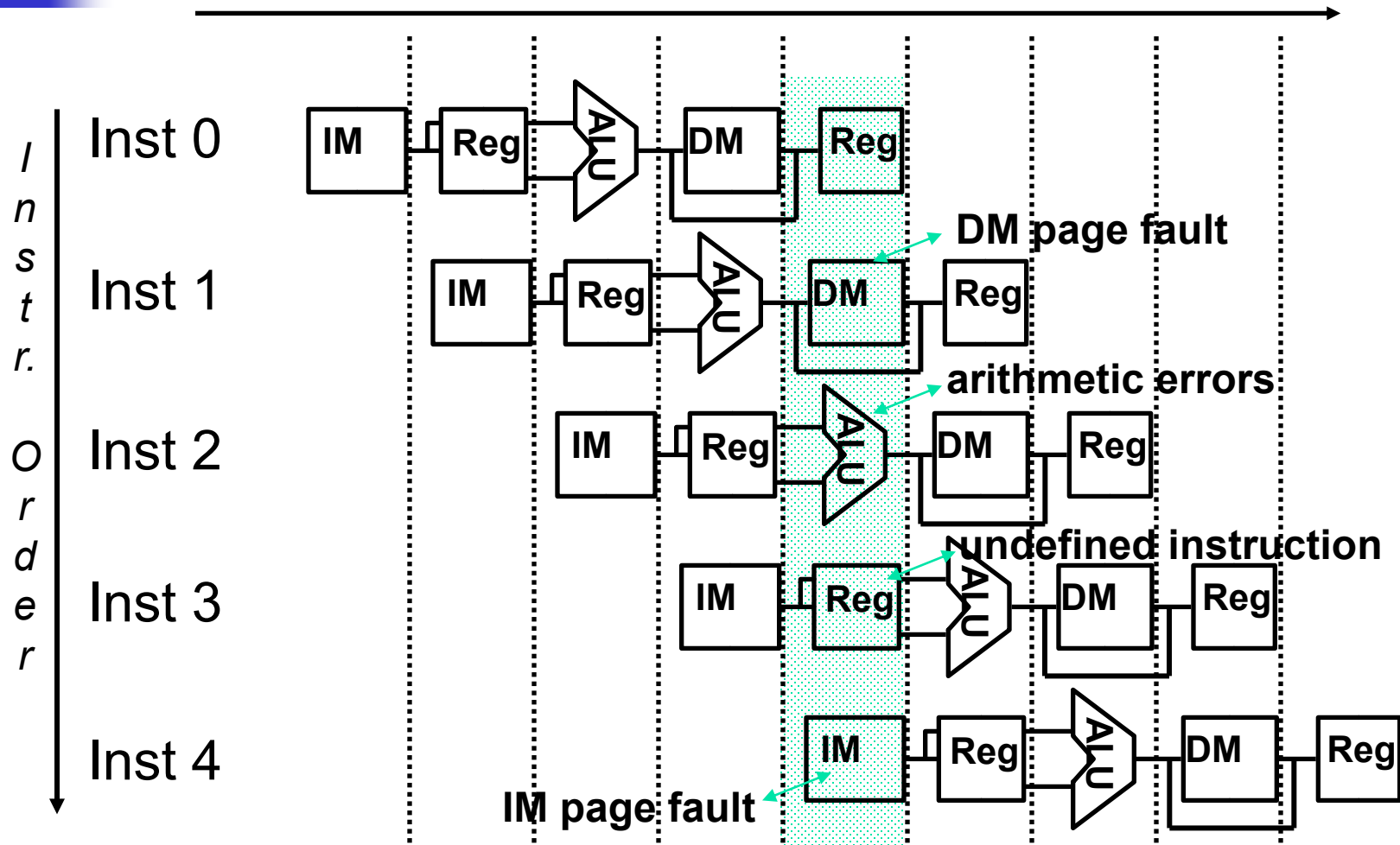
Two Types of Exceptions

- Interrupts – asynchronous to program execution
 - caused by **external events**
 - may be handled **between** instructions, so can let the instructions currently active in the pipeline *complete* before passing control to the OS interrupt handler
 - simply suspend and resume user program
 - `mcause[31] = 0` (`mcause` is positive) for interrupts
- Traps (Exceptions) – synchronous to program execution
 - caused by **internal events**
 - condition must be remedied by the trap handler for **that** instruction, so must stop the offending instruction *midstream* in the pipeline and pass control to the OS trap handler
 - the offending instruction may be retried (or simulated – can be used to support instructions / features that the hardware doesn't) and the program may continue or it may be aborted
 - `mcause[31] = 1` (`mcause` is negative) for exceptions

Where in the Pipeline Exceptions Occur

	Stage(s)?	Synchronous?
■ Arithmetic error	E	yes
■ Undefined instruction	D	yes
■ TLB or page fault	F, M	yes
■ I/O service request	any	no
■ Hardware malfunction	any	no
■ Beware that multiple exceptions can occur simultaneously in a single clock cycle		

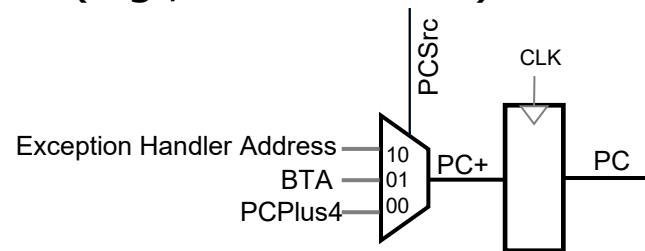
Multiple Simultaneous Exceptions



- Hardware sorts the exceptions so that the earliest instruction is the one interrupted first

Exception Handling Hardware

- Need a mechanism to load the PC with the address of the exception handler.
 - One way is to expand the PC input mux where the new input is
 - A fixed exception handler address (direct / non-vectorized interrupt)
 - Simpler hardware, but more complicated handler software as it needs to find the cause (e.g., from mcause) and deal with it appropriately



Control unit asserts
PCSrc = 10 when an interrupt /
exception occurs
(note: just to illustrate the idea.
The exact control signal
depends on the exact design)

- Or obtained from a table that has the handler addresses for various interrupts / exceptions (vectored interrupt)
 - More complicated hardware, but potentially simpler interrupt handler software, faster interrupt handling
- If `mtvec[0] == 0`, RV CPU uses direct mode, else vectored mode
 - In direct mode, `mtvec` used as the handler address directly.
Synchronous exceptions in RV always use direct mode (unlike ARM)
 - In vectored mode, `mtvec` is used as the base address of a table indexed by the interrupt ID (similar to VTOR in ARM)

Exception Handling Hardware

- Need to flush offending instruction and the ones that follow it in case of exceptions (flush may be not needed for interrupts*)
- Some processors automatically save caller-saved registers to the stack (ARM does this)
 - This is not automatic for RV. Requires some compiler support / inline assembly to save caller-saved registers
- Save either the instruction address or return address to a register (e.g., PC to `mepc` in RV*, PC+4 to LR in ARM)
- In some processors, there are separate (banked) copies of some registers for use by the handler, which reduces the amount of data to be saved / restored and eliminates the possibility of corruption / data loss (ARM does it for some registers such as SP; not done in RV)
- Interrupt processing can be complicated if there are many interrupt sources. Many systems include an interrupt controller - a separate unit closely coupled to the processor which contains control registers for enabling / disabling interrupts, setting priority of interrupts etc. e.g., NVIC in ARM, platform-level interrupt controller (PLIC) in RV

Exception Handler (Software) Steps

- Save additional status / registers etc. if needed
- Check which peripheral (if there are multiple interrupts with the same id) / condition within the peripheral caused the interrupt
 - Done by reading interrupt status registers within the peripheral
- Do the BARE MINIMUM required to deal with the interrupt
 - e.g., read the new data and store it in a buffer
- Clear the interrupt – tell the interrupting peripheral to de-assert the interrupt request
 - The way it is done is peripheral specific and might also depend on the condition within the peripheral which caused the interrupt
- To do further processing, set a Boolean flag* for the main program / wakeup a thread (bottom half of the handler)
- Restore additional status / registers etc if any
- Exit the Handler (using `mret` instruction which does `PC=mepc`)
 - Increment `mepc` before this if the instruction that caused the synchronous exception is not to be repeated (not applicable for interrupts)

Processor Modes in RV: Machine (M) Mode

- Processors have different operating mode with different privilege levels
- Machine (M) mode is the highest and most privileged mode, intended for low-level firmware (e.g., bootloaders, security monitors, hypervisors). It has full access to the system, including all CSRs (Control and Status Registers), physical memory, and the ability to configure other modes
- Controls hardware initialization, manages interrupts, and configures system-wide parameters
- Always available on any RV implementation, even embedded systems
- Has access to all CSRs, such as `mstatus`, `mcause`, `mepc`, `mtval`, `mtvec`
- Can delegate exceptions and interrupts to S-mode via `medeleg` and `mideleg` registers
- `mret` instruction returns to the privilege level from which the interrupt / exception occurred (M or S or U mode, remembered by MPP field of `mstatus` register. MPP = machine previous privilege)

RV: Supervisor (S) Mode

- Supervisor (S) Mode is used by operating systems. Provides a balance of access and restriction: the OS can manage resources while remaining constrained from machine-level hardware manipulation
- Manage system resources, memory, I/O for user processes
- Only available if the CPU is configured with RV privileged (S) extensions (e.g., for systems running an OS)
- Uses CSRs such as `sstatus`, `scause`, `sepc`, `stval` which are S-mode equivalents of the corresponding M-mode registers
- `satp` contains the page table address, allowing virtual memory management
- S-mode interrupts / exceptions have a lower priority than M-mode
- `ecall` instruction causes an M-mode exception. Any M-mode exception causes transition to M-mode
- `sret` instruction returns to the privilege level from which the interrupt / exception occurred (S or U mode)
- Attempt to write to a non-accessible register = illegal instruction exception, escalates to M-mode

RV: User (U) Mode

- User Mode is the lowest privilege level, intended for running application software. This mode has restricted access to system resources, isolating user programs from critical system components to enhance security and stability
- Executes unprivileged applications in a sandboxed environment, limiting their ability to affect the OS or hardware
- Available in systems with S-mode, works in tandem with S-mode to enforce isolation
- Has very limited access to CSRs, mainly for user-mode control
- `ustatus` (if implemented) holds status flags relevant to U-mode
- U-mode uses `ecall` to request services from the OS in S-mode
- Uses page tables managed by S-mode to restrict access to certain memory regions, enabling process isolation and memory protection
- U-mode usually cannot directly handle hardware interrupts or exceptions, relying instead on the OS (S-mode) for these services
- Attempt to write to a non-accessible register = illegal instrn exception, escalates to M-mode, but often handled by S-mode via delegation



Summary

Also read up about load-reserved/store-conditional instructions which are used for synchronisation in hardware or software multithreading
Hardware threads in RV are called harts

- Interrupt vs Exceptions – potential differences in handling
- Exception handling hardware – mechanism to change PC
- Modern processors have different modes and privilege levels
- Suggested reading : Priorities of interrupts and exceptions in RV
 - Timer Interrupt > External Interrupt > Software Interrupt > Exception
 - M-mode > S-mode > U-mode